POCHI

Patient Oriented Care & Health Intelligence

MANUAL DEL OPERADOR

Guía técnica de electrónica, programación y operación

Documento para el personal técnico que opera y mantiene a Pochi. Estado al 6 de octubre de 2026.

Índice

Aviso de seguridad — Lea antes de trabajar con Pochi 4

1. Introducción 5
- Qué cubre este manual 5
- Convenciones 5

2. Arquitectura general 5
- Decisiones de diseño 6

3. Hardware y electrónica 7
- Componentes principales 7
- Conexiones del ESP32 (pinout) 7
- Alimentación 8
- Comunicaciones 10
- Actuadores 10
- Sensores 11
- La pulsera 11
- Audio 11

4. Versiones de Pochi: para qué sirve cada una 12
- 4.1 Pochi normal 12
- 4.2 Pochi show 12
- 4.3 Pochi LiDAR 13
- 4.4 Herramientas manuales 13

5. Firmware del ESP32 13
- 5.1 Cómo cargar un firmware 13
- 5.2 Firmware normal (por cable) 14
- 5.3 Firmware WiFi (show) 14
- 5.4 Comportamiento común a los dos firmwares 15
- 5.5 Protocolo de comandos 15

6. Programa de la Raspberry 16
- 6.1 Ubicación y arranque 16
- 6.2 Mapa de archivos 16
- 6.3 Ciclo de entrega 18
- 6.4 Rastreo: “la pulsera es la estrella” 18
- 6.5 Protocolo de energía y brillo 19
- 6.6 Chat, voz y escucha 20
- 6.7 Base de datos 20
- 6.8 Interfaz y accesos ocultos 20

7. Operación diaria 21
- 7.1 Encendido 21
- 7.2 Revisión antes de cada día de uso 21
- 7.3 Carga de medicamentos 22
- 7.4 Apagado 22
- 7.5 Carga de baterías 22

8. Calibración y diagnóstico 22
- 8.1 Calibrar la pulsera 22
- 8.2 Diagnosticar el micrófono 23
- 8.3 Ver el LiDAR en la laptop 23
- 8.4 Probar los movimientos a mano 23

9. Preparación del show 23

10. Solución de problemas 24

11. Mantenimiento técnico 25

12. Estado del desarrollo (6 de octubre de 2026) 26
- Terminado 26

Validado solo en simulación 26
Pendiente 26
Anexo A. Referencia rápida de comandos 27
Anexo B. Glosario 27

Aviso de seguridad — Lea antes de trabajar con Pochi

Este manual es para la persona que opera, prueba y mantiene a Pochi: explica cómo está hecho por dentro (electrónica y programación), cómo se enciende, qué hace cada versión del programa y cómo resolver los problemas más comunes. Para el uso diario con el paciente existe el **Manual de Usuario**, que no incluye ningún detalle técnico.

Cargue cada batería por separado

Pochi tiene dos baterías de plomo-ácido de 12 V. Se cargan con el cargador Steren BR-700, una a la vez. Nunca conecte las dos baterías unidas al cargador.

La Raspberry se alimenta directo por los pines de 5 V

El puerto USB-C de la Raspberry Pi 5 está roto. La placa recibe 5 V por los pines GPIO, sin la protección de entrada que tiene el USB-C. Revise la polaridad antes de conectar, no conecte ni desconecte esa alimentación con el sistema encendido y no suba el convertidor por encima de unos 5.1 V.

Los brazos se mueven al abrir el puerto del ESP32

Abrir el puerto serial del ESP32 lo reinicia, y al reiniciar energiza los servos de los brazos de golpe. Mantenga los motores apagados en lo que se reinicia la esp32. Mantenga las manos lejos de los brazos al arrancar `main.py`, `panel_control.py` o el Monitor Serie.

Un solo programa a la vez

Pochi normal, Pochi show, Pochi LiDAR y el panel de control por cable nunca deben estar abiertos al mismo tiempo. Los programas lo revisan, pero siempre cierre uno antes de abrir el otro.

Reglas que no se cambian

Pochi no dispensa nada sin que el paciente toque “Sí, la voy a tomar”. El chat de voz solo conversa: no controla el robot ni da consejos médicos (`PERMITIR_ORDENES_DESDE_CHAT = False`). El color de cada medicamento en el panel debe coincidir con el compartimento físico.

1. Introducción

POCHI (Patient Oriented Care & Health Intelligence) es un robot asistente de medicamentos para personas adultas mayores, desarrollado como proyecto de feria científica del Colegio Capouilliez, Guatemala, 2026. A la hora programada, Pochi:

1. Busca físicamente al paciente en la habitación, guiándose por una pulsera Bluetooth (BLE) que el paciente lleva puesta.
2. Le pregunta en pantalla y en voz alta si va a tomar su medicamento.
3. Si responde que sí, dispensa la pastilla del compartimento de color correcto y la entrega en un plato que suben dos brazos.
4. Espera la confirmación "Ya la tomé" y guarda un historial de las tomas.

Además tiene un chat con voz que sirve solo para hacer compañía.

Qué cubre este manual

- La arquitectura general y cómo se reparten el trabajo la Raspberry Pi y el ESP32.
- El hardware: piezas, conexiones del ESP32, alimentación y comunicaciones.
- Las versiones de Pochi (normal, show y LiDAR) y para qué sirve cada una.
- El firmware del ESP32 y su protocolo de comandos.
- El programa de la Raspberry: archivos, ciclo de entrega, rastreo, energía, voz y base de datos.
- Procedimientos de operación, calibración, preparación del show y solución de problemas.

Convenciones

- El texto con `este formato` es un comando, un archivo o una constante del código, escrito exactamente como aparece.
- Las rutas de archivos son relativas a la carpeta del programa, `/home/temario/Documents/interfaz_ia_sistema`, salvo que se indique otra.
- Los recuadros grises con `[ Agregar aquí … ]` marcan datos que el equipo todavía debe completar.
- Cuando una función solo se ha probado en simulación y no en el robot real, el manual lo dice.

2. Arquitectura general

Pochi usa una arquitectura **maestro–esclavo**. La **Raspberry Pi 5 decide** todo (horarios, rastreo, pantalla, voz e historial) y el **ESP32 ejecuta** las órdenes físicas (orugas, brazos, dispensadores y sensores).

```
                 USB serial (texto, 115200)
Raspberry Pi 5  ──────────────────────────►  ESP32 (orugas, brazos,
(decide todo)   ◄── LOG: / SYS_STATUS: ──    dispensadores, sensores)
     │
     ├── USB 230400 ──────────── LiDAR LD06
     ├── Bluetooth (BLE) ◄──── Pulsera del paciente
     └── UDP 5005 (broadcast) ─► Laptop: visor_lidar.py
```

Componente	Rol	Se comunica por
Raspberry Pi 5	Decide todo: horarios, rastreo, pantalla, voz, chat e historial.	USB serial al ESP32, USB al LiDAR, Bluetooth con la pulsera, UDP a la laptop
ESP32	Ejecuta órdenes físicas. Hace el ciclo completo de dispensación por sí solo.	USB serial (115200)
LiDAR LD06	Mide obstáculos en 360° para esquivarlos.	USB (230400)
Pulsera	Emite una señal BLE cada 100 ms para que Pochi sepa dónde está el paciente.	BLE (la recibe la Raspberry)
Laptop (opcional)	Muestra el radar del LiDAR en vivo.	UDP, puerto 5005
Teléfono (solo show)	Maneja orugas y brazos a distancia.	WiFi / WebSocket al ESP32 (firmware WiFi)

Decisiones de diseño

Estas decisiones ya están tomadas. Conviene entender el porqué antes de modificar algo.

Decisión	Por qué
La dispensación está 100 % en el ESP32	La Raspberry solo manda `SYS_START_<color>_<dosis>` y el ESP32 hace todo el ciclo. Si la comunicación falla a mitad de camino, la entrega no queda a medias.
El puerto del ESP32 se abre una sola vez	Abrir el puerto reinicia el ESP32 y energiza los brazos de golpe. Por eso se abre al arrancar y se queda abierto hasta cerrar el programa.
En el rastreo manda la pulsera	La pulsera indica hacia dónde ir; el LiDAR solo sirve para esquivar. Así Pochi no se acerca a la primera figura que ve.
Energía: "una cosa a la vez"	Evita picos de consumo que reinician la Raspberry. Solo orugas y LiDAR funcionan juntos.
No dispensa sin confirmación	Nada sale del dispensador hasta que el paciente toca "Sí, la voy a tomar".
El chat solo conversa	No controla el robot ni da consejos médicos.
En el show, la Raspberry no habla con el ESP32	El ESP32 lo maneja el teléfono por WiFi; la Raspberry solo lo alimenta por USB.
Interfaz oscura y minimalista	El fondo blanco molesta a la vista del adulto mayor.

3. Hardware y electrónica

Componentes principales

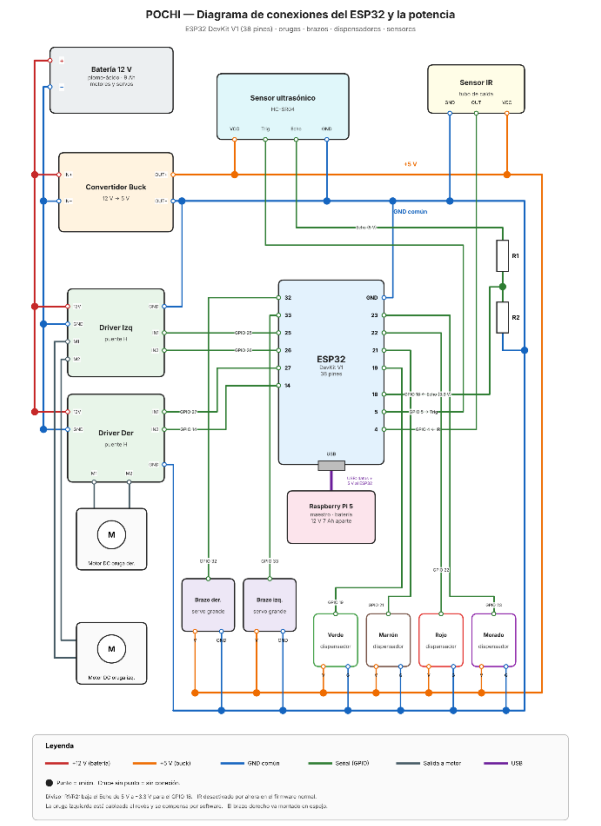
Pieza	Modelo / detalle	Función
Cerebro (maestro)	Raspberry Pi 5, 4 GB. Python 3 + PyQt5	Programa principal, interfaz, decisiones
Pantalla	Táctil de 7", 800 × 480	Cara de Pochi, alertas, panel de administración, chat
Músculo (esclavo)	ESP32 DevKit V1, 38 pines. Firmware en PlatformIO (Arduino)	Mueve orugas, brazos y dispensadores; lee sensores
LiDAR	LD06, 360°, por USB	Detecta obstáculos durante el rastreo
Orugas	2 motores DC	Desplazamiento
Brazos	2 servos	Suben y bajan el plato con la pastilla
Dispensadores	4 servos de giro continuo, uno por color	Sueltan la pastilla del compartimento
Sensores	Ultrasónico e infrarrojo en el tubo de caída	Freno ante obstáculos y detección de la pastilla
Audio	Bocina USB y micrófono USB	Voz de Pochi y chat por voz
Pulsera	ESP32 aparte que emite señal BLE; carga por microUSB	Permite ubicar al paciente
Energía	2 baterías de plomo-ácido selladas de 12 V; convertidor 12 V → 5 V; cargador Steren BR-700	Alimentación de todo el robot

Datos físicos registrados: Pochi pesa unos **15 kg**, se apoya en 4 ruedas de 5 cm en cruz a 17.5 cm del centro y el engranaje motriz mide 5 cm de diámetro. El LiDAR mide desde el centro del robot y el cuerpo ocupa unos **25 cm de radio** alrededor de ese punto.

Conexiones del ESP32 (pinout)

GPIO	Conectado a	Notas
26 / 25	Motor de oruga (par 1)	Un motor de oruga por par de pines.
14 / 27	Motor de oruga (par 2)	La oruga izquierda está cableada al revés; se compensa por software.
33	Servo del brazo izquierdo	
32	Servo del brazo derecho	Montado en espejo: el mismo pulso lo mueve en sentido contrario.
19	Dispensador verde (V)	Servo de giro continuo
21	Dispensador marrón (M)	Servo de giro continuo

GPIO	Conectado a	Notas
22	Dispensador rojo (R)	Servo de giro continuo
23	Dispensador morado (P)	Servo de giro continuo
5	Ultrasónico — Trig	
18	Ultrasónico — Echo	
4	Infrarrojo del tubo de caída	Detecta que la pastilla cayó.



Alimentación

Batería	Alimenta
12 V · 9 Ah (principal)	Motores de las orugas y servos (brazos y dispensadores).
12 V · 7 Ah (lógica)	Raspberry Pi 5, a través del convertidor 12 V → 5 V conectado a los pines GPIO de 5 V. La Raspberry, a su vez, alimenta por USB al ESP32, al LiDAR, a la bocina y al micrófono, y alimenta la pantalla.

```
Batería 12 V · 9 Ah ──► motores de orugas + servos de brazos y dispensadores

Batería 12 V · 7 Ah ──► convertidor 12 V→5 V (~4.99 V) ──► GPIO 5 V de la Raspberry
                                                          ├─ USB ─► ESP32 (lógica)
                                                          ├─ USB ─► LiDAR LD06
                                                          ├─ USB ─► bocina y micrófono
                                                          └─ DSI ─► pantalla táctil
```

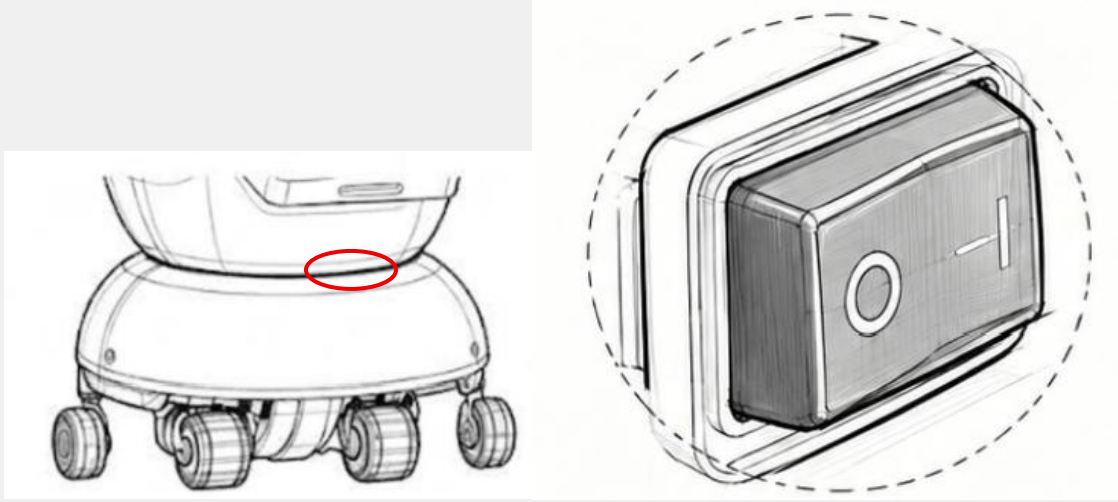
Problema de energía conocido (el más importante)

Pochi se ha reiniciado o cerrado al subir los brazos o al sonar la bocina, y aparece "Low voltage warning" al usar el micrófono. Causas: el convertidor da ~4.99 V (justo en el límite) y la Raspberry entra por los pines GPIO, sin la protección del USB-C. Además, la Raspberry Pi 5 limita la corriente total de sus puertos USB a 600 mA (1.6 A con `usb_max_current_enable=1`, que ya está puesto en `config.txt`).

El software ya reduce los picos con el protocolo de energía (sección 6.5), pero **la solución de raíz es de hardware**:

- Ajustar el convertidor de forma estable a unos **5.1 V**.
- Usar cables de alimentación **cortos y gruesos**.
- O conectar el LiDAR y el ESP32 a un **hub USB con alimentación propia**, para no depender de los puertos de la Raspberry.

Para revisar si hubo bajo voltaje, ejecute en la Raspberry `vcgencmd get_throttled`. El resultado `throttled=0x0` significa que no hubo problemas; cualquier otro valor indica bajo voltaje o limitación. Conviene revisarlo antes y después de mover los brazos o activar el LiDAR.



La batería de la Raspberry con su interruptor se encuentra adentro en la parte inferior derecha con el interruptor apuntando hacia el frente.

Comunicaciones

Enlace	Entre	Velocidad / puerto	Detalle
USB serial	Raspberry ↔ ESP32	115200 baudios	Chip CP2102N (VID 10C4, PID EA60). Se busca en `/dev/serial/by-id` con el patrón `*CP2102N*`. El mismo cable alimenta al ESP32.
USB	Raspberry ↔ LiDAR	230400 baudios	Chip CP2102 (sin N). Patrón `*CP2102_*`. Solo se enciende durante el rastreo.
BLE	Pulsera → Raspberry	Anuncio cada 100 ms	MAC `B0:CB:D8:C6:A5:5E`. Librería `bleak` en `robot_movimiento.py`.
UDP	Raspberry → laptop	Puerto 5005, broadcast	Mapa del LiDAR para `visor_lidar.py`. Ambos equipos en la misma red.
WiFi / WebSocket	Teléfono ↔ ESP32	Ruta `/ws`, red de 2.4 GHz	Solo con el firmware WiFi (show). Página en `http://pochi.local` o en la IP del ESP32.

> **Por qué importa el "by-id"**
> El ESP32 y el LiDAR usan chips USB casi iguales (CP2102N y CP2102). El programa los distingue por su nombre en `/dev/serial/by-id`, no por `/dev/ttyUSB0` o `/dev/ttyUSB1`, que pueden cambiar de orden en cada arranque.

Actuadores

Orugas

Dos motores DC. La oruga izquierda está cableada al revés y se compensa por software. Para ahorrar energía, el firmware libera las orugas 2 s después de detenerse.

Brazos

Dos servos que se mueven de forma gradual (pasos de 10 µs cada 12 ms; unos 0.5 s por recorrido), siempre dentro de su rango calibrado. Pueden moverse juntos o por separado. Se desconectan 1.5 s después de bajar para ahorrar energía.

Brazo	Reposo (µs)	Arriba (µs)
Izquierdo (GPIO 33)	1232	1639
Derecho (GPIO 32, en espejo)	1511	1118

Al volver a conectar un servo, el firmware escribe primero su posición real. Esto evita el "brinco" que causaba la librería ESP32Servo, que manda 1500 µs al hacer `attach()`.

Dispensadores

Cuatro servos de giro continuo, uno por color: verde (V), marrón (M), rojo (R) y morado (P). Solo reciben energía mientras giran.

Sensores

Sensor	Uso	Estado actual
LiDAR LD06	Mapa de obstáculos en 360° para esquivar durante el rastreo.	Activo. Solo se enciende durante el rastreo (en Pochi normal).
Ultrasónico	Distancia al frente. En el firmware WiFi frena las orugas si hay algo a menos de 30 cm (`FRENO_OBSTACULO`).	Activo.
Infrarrojo del tubo de caída	Confirma que la pastilla cayó antes de subir los brazos.	**Desactivado en el firmware normal**: los brazos suben solos 10 s después de `SYS_START`. Activo en el firmware WiFi.

La pulsera

Es un ESP32 independiente que solo emite anuncios BLE cada 100 ms. Su MAC es `B0:CB:D8:C6:A5:5E` y está escrita en `robot_movimiento.py` (`TARGET_MAC`). Se carga por microUSB; cuando está encendida muestra unas tiras de luces rojas en su pantalla, encima del símbolo de batería.

La pulsera solo entrega la **intensidad de la señal (RSSI)**, no la dirección. Pochi calcula la dirección girando sobre sí mismo (sección 6.4).

> **Si se cambia la pulsera o su firmware**
> Lea la MAC nueva en el Monitor Serie de la pulsera y actualice `TARGET_MAC` en `robot_movimiento.py`. Después vuelva a calibrar la pulsera (sección 8.1).

- Evitar mojarla o exponerla a golpes fuertes.
- Cargarla con regularidad para asegurar que Pochi pueda detectarla durante todo el día.
- Batería de la pulsera: 6 horas
- Tiempo de carga: 1 día

Audio

Bocina USB y micrófono USB conectados a la Raspberry. Antes se usaba un altavoz-micrófono Yealink; los archivos de esa versión (`ia/*(yealinks).py`) ya no se usan.

4. Versiones de Pochi: para qué sirve cada una

Pochi tiene **tres programas** en la Raspberry y **dos firmwares** para el ESP32. Cada combinación sirve para algo distinto.

Versión	Programa en la Raspberry	Firmware del ESP32	¿Quién maneja al ESP32?	Para qué sirve
Pochi normal	`main.py` (autoarranque)	Normal (por cable)	La Raspberry, por USB	Uso real con el paciente: todo automático.
Pochi show	`modo_show.py` o ícono "POCHI Show"	WiFi	El teléfono, por WiFi. La Raspberry nunca le habla; solo lo alimenta.	Presentaciones en público.
Pochi LiDAR	`modo_lidar.py` o ícono "POCHI LiDAR"	No se usa	Nadie	Pruebas del LiDAR y del visor en la laptop.
Panel por cable	`panel_control.py` (con Pochi cerrado)	Normal	El operador, desde la pantalla	Probar a mano brazos, orugas y dispensadores.

Nunca abra dos versiones a la vez

Antes de abrir el modo show, el modo LiDAR o el panel por cable, detenga Pochi normal con `systemctl --user stop pochi`. Para volver a Pochi normal, cierre el modo que esté abierto y ejecute `systemctl --user start pochi`, o reinicie la Raspberry.

4.1 Pochi normal

Es la versión para el paciente. Se abre sola al encender si el autoarranque está instalado (servicio systemd de usuario `pochi`), o a mano con `python3 main.py`. Funciona todo: horarios, rastreo con pulsera y LiDAR, dispensación real, voz, chat, historial y ahorro de energía.

- Requiere el **firmware normal** en el ESP32, conectado por USB.
- Se cierra con el botón ✕ rojo arriba a la derecha (cierre limpio) o con `systemctl --user stop pochi`.
- Si no encuentra el ESP32 o el LiDAR, el programa entra en modo simulación y sigue funcionando en pantalla. Revise los mensajes de arranque para confirmar que encontró el hardware.

4.2 Pochi show

Pensado para presentaciones. En la pantalla todo se ve y funciona como en Pochi normal (horarios, alertas, preguntas, confirmaciones), pero la Raspberry **nunca se comunica con el ESP32**:

- La dispensación es simulada, con tiempos reales (unos 11.5 s).
- El rastreo es simulado (unos 3 s).

- Las orugas y los brazos se manejan desde un **teléfono** conectado por WiFi al ESP32, que debe tener cargado el **firmware WiFi**.
- La Raspberry sigue alimentando al ESP32 por el cable USB.

Se abre con el ícono de escritorio "POCHI Show" o con `python3 modo_show.py`. La preparación completa está en la sección 9.

4.3 Pochi LiDAR

Programa de prueba: muestra los ojos de Pochi y mantiene el LiDAR **siempre encendido**, enviando el mapa al visor de la laptop (`visor_lidar.py`). No usa el ESP32. Sirve para revisar el LiDAR, ver cómo "ve" Pochi el lugar y mostrarlo en una presentación. Se abre con el ícono "POCHI LiDAR" o con `python3 modo_lidar.py`.

4.4 Herramientas manuales

- **Panel de control por cable** (`panel_control.py`): botones en la pantalla para brazos, orugas y dispensadores. Solo se usa con Pochi cerrado, porque ocupa el mismo puerto del ESP32.
- **Panel Web Serial** (`index.html` dentro de la carpeta del programa): panel por cable que se abre en un navegador compatible con Web Serial (por ejemplo Chrome) en una computadora conectada por USB al ESP32. No es la página del teléfono.
- **Página del teléfono** (`data/index.html` del firmware WiFi): la sirve el propio ESP32 en el show.

Los íconos de escritorio "POCHI Show" y "POCHI LiDAR" se instalan con `modos/instalar_iconos_modos.sh` y usan `modos/lanzar_modo.sh`. Si el instalador da "Permission denied", ejecútelo con `bash modos/instalar_iconos_modos.sh`.

5. Firmware del ESP32

El firmware está hecho en **PlatformIO** (framework Arduino). Cada firmware es un proyecto con su código en `src/main.cpp`. Hay dos: el **normal** (por cable, para el paciente) y el **WiFi** (para el show).

Firmware se encontrara en la usb de copia de seguridad con todos los códigos

5.1 Cómo cargar un firmware

1. Si el ESP32 está conectado a la Raspberry, cierre Pochi (`systemctl --user stop pochi`). El puerto no puede estar ocupado por otro programa.
2. Conecte el ESP32 por USB a la computadora donde está PlatformIO (o abra PlatformIO en la Raspberry).
3. Abra el proyecto del firmware que necesita (normal o WiFi) en VS Code con PlatformIO.
4. Pulse **Build** y revise que compile sin errores.
5. Pulse **Upload** para cargar el firmware.

6. Solo para el firmware WiFi: pulse también **Upload Filesystem Image**, que sube la página del teléfono (`data/index.html`).
7. Abra el **Monitor Serie** a **115200** baudios para ver los mensajes de arranque (y la IP, en el firmware WiFi).
8. Vuelva a conectar el ESP32 a la Raspberry y abra el programa que corresponda.

> **Cuidado con los brazos**
> Cargar el firmware o abrir el Monitor Serie reinicia el ESP32 y los brazos pueden moverse de golpe. Mantenga las manos alejadas.

5.2 Firmware normal (por cable)

- Recibe los comandos de la Raspberry por USB.
- Hace el ciclo completo de dispensación cuando recibe `SYS_START_<color>_<dosis>`.
- **El infrarrojo está desactivado temporalmente**: los brazos suben solos 10 s después de `SYS_START`. Cuando se reinstale el sensor hay que restaurar esa lógica (se puede tomar del firmware WiFi).

5.3 Firmware WiFi (show)

- Sirve la página del teléfono (`data/index.html`) y recibe comandos por **WebSocket** en `/ws`. También acepta comandos por USB.
- **Red:** el nombre y la contraseña están escritos en el código. El ESP32 y el teléfono deben estar en la **misma red WiFi de 2.4 GHz**.
- La IP aparece en el Monitor Serie (115200). También se puede abrir `http://pochi.local`.
- Si la red no está disponible al encender, el ESP32 arranca igual y se conecta solo cuando la red aparece.
- Tiene la lógica completa del infrarrojo: basta una detección; luego espera 6 s de asentamiento y 4 s más antes de subir los brazos. Si en 10 s no detecta nada, reporta `TIMEOUT`.

Seguridad de las orugas en el firmware WiFi

Las orugas se detienen solas en cualquiera de estos casos:

- El teléfono se desconecta o se cae el WiFi.
- Dejan de llegar las repeticiones de la orden durante 1.5 s. La página repite la orden cada 0.3 s mientras se mantiene presionado el botón (seguro de “hombre muerto”): al soltar el botón, Pochi se detiene.
- Al avanzar, el ultrasónico detecta algo a menos de 30 cm (`FRENO_OBSTACULO`).

5.4 Comportamiento común a los dos firmwares

- Brazos con movimiento gradual, limitados a su rango calibrado e independientes.
- Al reconectar un servo se escribe primero su posición real (evita el brinco).
- Ahorro de energía: brazos desconectados 1.5 s después de bajar, orugas liberadas 2 s después de detenerse, dispensadores con energía solo mientras giran.
- Mientras hay una dispensación activa, el ESP32 **ignora los comandos manuales**.

5.5 Protocolo de comandos

La comunicación es por texto, **una orden por línea**. Los colores se escriben con una letra: `V` verde, `M` marrón, `R` rojo, `P` morado.

Comandos manuales

Comando	Qué hace
`ARM_L_UP` / `ARM_L_DN`	Sube / baja el brazo izquierdo.
`ARM_R_UP` / `ARM_R_DN`	Sube / baja el brazo derecho.
`ARM_BOTH_UP` / `ARM_BOTH_DN`	Sube / baja los dos brazos.
`MOT_F` / `MOT_B`	Orugas hacia adelante / hacia atrás.
`MOT_L` / `MOT_R`	Gira a la izquierda / a la derecha.
`MOT_S`	Detiene las orugas.
`DISP_TEST_V` / `_M` / `_R` / `_P`	Prueba el dispensador de ese color.
`SENS_US_ON` / `SENS_US_OFF`	Activa / desactiva el reporte del ultrasónico.
`SENS_IR_ON` / `SENS_IR_OFF`	Activa / desactiva el reporte del infrarrojo.

Ciclo automático de dispensación

Comando	Qué hace
`SYS_START_<color>_<dosis>`	Inicia el ciclo completo. Ejemplo: `SYS_START_V_1` = 1 pastilla del compartimento verde.
`SYS_RETRY`	Reintenta la dispensación.
`SYS_RECEIVED`	El paciente confirmó “Ya la tomé”: termina el ciclo y baja los brazos.
`SYS_POSTPONE_DN` / `SYS_POSTPONE_UP`	Baja / vuelve a subir los brazos en un aplazamiento.
`SYS_STOP`	Detiene el ciclo.

Respuestas del ESP32

Respuesta	Significado
`LOG:…`	Mensaje informativo (para diagnóstico).
`SYS_STATUS:WAIT_IR`	Esperando que la pastilla pase por el infrarrojo.
`SYS_STATUS:DETECTED_IR`	La pastilla fue detectada.
`SYS_STATUS:TIMEOUT`	No se detectó la pastilla a tiempo.
`SYS_STATUS:ARMS_UP`	Brazos arriba: la pastilla está servida.
`SYS_STATUS:ENDED`	Ciclo terminado.
`SYS_STATUS:POSTPONED`	Ciclo aplazado.
`SYS_STATUS:STOPPED`	Ciclo detenido.
`SENSOR_US:…` / `SENSOR_IR:…`	Lecturas del ultrasónico y del infrarrojo.

6. Programa de la Raspberry

6.1 Ubicación y arranque

El programa está en `/home/temario/Documents/interfaz_ia_sistema` y usa el entorno virtual `entorno/` (creado con `--system-site-packages`). Para abrirlo a mano:

```
cd /home/temario/Documents/interfaz_ia_sistema
source entorno/bin/activate
python3 main.py
```

Si el autoarranque está instalado, Pochi abre solo al encender mediante el servicio systemd de usuario `pochi`:

```
systemctl --user stop pochi       # detener Pochi
systemctl --user start pochi      # volver a abrir Pochi normal
systemctl --user restart pochi    # reiniciar el programa
systemctl --user status pochi     # ver si está corriendo
journalctl --user -u pochi -f     # ver los mensajes del programa en vivo
```

6.2 Mapa de archivos

Archivo	Qué hace
`main.py`	Punto de entrada: crea la pantalla, arranca el scheduler, maneja los accesos ocultos, el cierre limpio (botón ×) y los toques que devuelven el brillo.
`scheduler.py`	Motor de entregas: revisa los horarios cada minuto y corre el ciclo de entrega (rastreo → pregunta → dispensar → confirmar).
`robot_movimiento.py`	Rastreo: pulsera BLE (bleak), LiDAR LD06, navegación; publica el mapa por UDP.

Archivo	Qué hace
`robot_esp32.py`	Comunicación serial con el ESP32: comandos, estados, esperas con tiempo límite y simulación fuera de la Raspberry.
`energia.py`	Protocolo de energía “una cosa a la vez” y brillo de la pantalla.
`database.py`	Base de datos SQLite `robot.db`: usuario, medicamentos, horarios, historial y alertas.
`ui/ventana_main.py`	Pantalla principal: ojos, hora, próximo medicamento e indicador de la pulsera.
`ui/ojos.py`	Cara animada: normal, feliz, alerta y dormido (de 22:00 a 6:00).
`ui/ventana_alerta.py`	Ventana de la toma: pregunta, preparando, servida, confirmación y fallo.
`ui/ventana_admin.py`	Panel del cuidador: Usuario, Medicamentos, Horarios e Historial; teclado táctil automático (matchbox-keyboard).
`ui/ventana_ia.py`	Chat con Pochi (hablar o escribir).
`ui/tactil.py`	Soporte de toques en la pantalla de 7".
`ia/cerebro.py`	Respuestas del chat con la API de Claude (Anthropic). `PERMITIR_ORDENES_DESDE_CHAT = False`.
`ia/voz.py`	Voz con ElevenLabs (respaldo: espeak), en una cola aparte; `callar()`; respeta el protocolo de energía.
`ia/escucha.py`	Reconocimiento de voz sin internet con Vosk (modelo en español). Constantes `NOMBRE_MICROFONO` y `GANANCIA_MIC`.
`robot_comandos.py`	Órdenes de movimiento por lenguaje natural (desactivado desde el chat).
`calibrar_pulsera.py`	Calibra la distancia y la dirección de la pulsera.
`probar_microfono.py`	Diagnóstico del micrófono.
`modo_show.py`	Versión para el show (sección 4.2).
`modo_lidar.py`	Versión de prueba del LiDAR (sección 4.3).
`modos/instalar_iconos_modos.sh`, `modos/lanzar_modo.sh`	Íconos de escritorio “POCHI Show” y “POCHI LiDAR”.
`panel_control.py`	Panel manual por cable. Solo con Pochi cerrado.
`index.html`	Panel por cable con Web Serial. No es el del teléfono.
`verificar_puertos.py`, `escaner.py`, `prueba_LIDAR.py`, `prueba_laptop.py`	Utilidades de prueba.
`main2.py`, `ia/*(yealinks).py`	Versiones viejas. No se usan.

Fuera de la Raspberry: `visor_lidar.py` corre en la laptop, y los dos firmwares con su `data/index.html` están en los proyectos de PlatformIO.

6.3 Ciclo de entrega

Lo maneja `scheduler.py`, que revisa los horarios cada minuto:

1. A la hora programada aparece la alerta y los ojos cambian a “alerta”.
2. **Si se detecta la pulsera**, Pochi sale a buscar al paciente (hasta 180 s; el scheduler espera 190 s). Si no la detecta, no se mueve y muestra la pregunta donde está.
3. Pregunta “¿Vas a tomarla ahora?”. **Sí** → dispensa. **Ahora no** o 5 min sin respuesta → aplaza 5 min. Máximo **3 rondas**; después se registra “Omitido”.
4. Dispensa (“Preparando tu pastilla…”) → brazos arriba (“Tu pastilla está lista”) → “Ya la tomé” baja los brazos y registra “Tomado”. Si nadie confirma en **60 s**, baja el plato solo.
5. Si el ESP32 no entrega, aparece “No pude darte tu pastilla” y se registra el fallo.
6. Si hay varios medicamentos a la misma hora, se encadenan sin volver a rastrear.

6.4 Rastreo: “la pulsera es la estrella”

La pulsera solo da intensidad de señal (RSSI). Para saber hacia dónde está, Pochi da una **vuelta de búsqueda** midiendo la señal y ajusta una **curva coseno** a toda la vuelta. En simulación, el error de dirección es de unos 5° a 15°.

- Pochi **sigue siempre la dirección de la pulsera**. El LiDAR solo sirve para esquivar: de los caminos libres, elige el más cercano a esa dirección.
- Si la señal cae 6 dB dos veces seguidas, vuelve a medir (hasta 3 veces).
- El giro se mide con el LiDAR, solo mientras Pochi gira (no tiene encoders en los motores).
- **Llega** cuando la pulsera marca 120 cm o menos (`DISTANCIA_DISPENSACION_CM`), haya lo que haya enfrente. Luego da una vuelta corta para quedar de frente a la pulsera (`ORIENTAR_AL_LLEGAR`).
- Al activar el LiDAR espera medio segundo de calentamiento antes de decidir, porque los primeros datos llegan vacíos.

Constantes que se pueden ajustar

Constante	Archivo	Valor	Si sube	Si baja
`TIMEOUT_SEG`	`robot_movimiento.py`	180 s	Busca más tiempo.	Se rinde antes y muestra la pregunta donde esté.
`SEGUNDOS_ACERCARSE`	`scheduler.py`	190 s	El scheduler espera más al rastreo.	Debe seguir siendo **mayor** que `TIMEOUT_SEG`, o cortará el rastreo.

Constante	Archivo	Valor	Si sube	Si baja
`DISTANCIA_DISPENSACION_CM`	`robot_movimiento.py`	120 cm	Se detiene más lejos del paciente.	Se acerca más; con mala calibración puede no "llegar" nunca.
`DISTANCIA_PARADA_CM`	`robot_movimiento.py`	55 cm	Se frena más lejos de lo que tiene delante.	Se acerca más a los obstáculos.
`RSSI_A_1M_DBM`, `EXPONENTE_TRAYECTO`	`robot_movimiento.py`	Calibrados	Los fija `calibrar_pulsera.py` (distancia).	
`OFFSET_DIRECCION_GRADOS`	`robot_movimiento.py`	Calibrado	Lo fija `calibrar_pulsera.py` (dirección).	
`TARGET_MAC`	`robot_movimiento.py`	`B0:CB:D8:C6:A5:5E`	MAC de la pulsera.	

> **La calibración de la pulsera es obligatoria**
> Si la distancia está mal calibrada, Pochi se detiene lejos o no se detiene. Si la antena de la pulsera "ve" mejor hacia un lado o hacia atrás, Pochi va en la dirección equivocada. Calibre en el lugar real donde se usará Pochi (sección 8.1).

Estado: este rastreo (pulsera como guía y vuelta de orientación al llegar) y la calibración de dirección se han validado **solo en simulación**. Falta probarlos en el robot real.

6.5 Protocolo de energía y brillo

`energia.py` aplica la regla "una cosa a la vez" para evitar picos de consumo:

Actividad	Regla
Orugas + LiDAR	Funcionan juntos (es la única excepción).
Brazos	Solos.
Dispensador	Solo.
Bocina	Solo con todo quieto. La frase se prepara antes y suena al terminar el movimiento.
Micrófono	Solo con todo quieto y la bocina callada.

El LiDAR solo se enciende durante el rastreo (lo controla `main.py`). El brillo de la pantalla cambia así:

Situación	Brillo
Mientras rastrea	15 %
2 minutos sin tocar la pantalla	30 %
Cualquier toque o alerta	100 %

El control de brillo solo funciona con la pantalla DSI y necesita una regla udev que se instala una sola vez.

Analizar el código de "energía.py", esta regla se mantiene privada del manual.

6.6 Chat, voz y escucha

Módulo	Cómo funciona	Necesita
`ia/cerebro.py`	Genera las respuestas del chat con la API de Claude. Solo conversa; `PERMITIR_ORDENES_DESDE_CHAT = False` no se debe cambiar.	Internet y clave de API de Anthropic
`ia/voz.py`	Convierte el texto en voz con ElevenLabs. Si falla, usa espeak. Funciona en una cola aparte y se puede interrumpir con `callar()`.	Internet y clave de ElevenLabs (espeak funciona sin internet)
`ia/escucha.py`	Reconoce la voz con Vosk, sin internet. Muestra "Preparando el micrófono…" y luego "Escuchando… habla ahora"; hay unos 8 s para hablar.	Modelo de Vosk en español; `NOMBRE_MICROFONO` y `GANANCIA_MIC` bien configurados

Claves de API de Anthropic y Elevenlabs están ocultas en el programa por privacidad del prototipo.

6.7 Base de datos

Todo se guarda en `robot.db` (SQLite), dentro de la carpeta del programa. Tiene los datos del usuario, los medicamentos, los horarios, el historial de tomas y las alertas. Para respaldarla, cierre Pochi y copie el archivo `robot.db` a otro lugar.

6.8 Interfaz y accesos ocultos

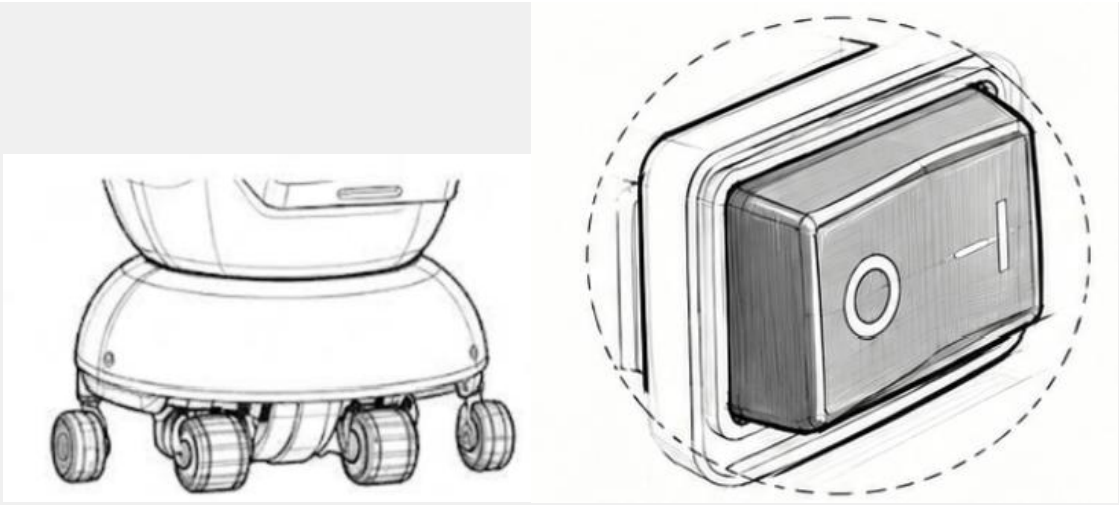
Acción en la pantalla principal	Resultado
3 toques en la esquina superior izquierda (en menos de 2 s)	Panel de administración
3 toques en la esquina inferior izquierda	Chat con Pochi
Botón ✕ arriba a la derecha	Cierra el programa de forma limpia

El uso del panel de administración y del chat está explicado en el Manual de Usuario.

7. Operación diaria

7.1 Encendido

1. Verifique que las dos baterías estén cargadas y conectadas.
2. Encienda a Pochi con su interruptor.
3. Espere a que la Raspberry arranque. Con el autoarranque, Pochi normal se abre solo.
4. Mantenga las manos lejos de los brazos: al abrir el puerto, el ESP32 se reinicia.
5. Confirme que aparece la pantalla principal y que el punto de la pulsera está en verde.



Ubicación botón de encendido de motores. La batería de la Raspberry con su interruptor se encuentra adentro en la parte inferior derecha con el interruptor apuntando hacia el frente.

Pasos de encendido:

1. *Encender batería de la raspberry*
2. *Esperar que el programa de Pochi encienda y se establezca con los ojos puestos*
3. *Encender batería de motores*

7.2 Revisión antes de cada día de uso

- Baterías cargadas.
- Pulsera cargada, encendida (luces rojas) y puesta en el paciente; punto verde en la pantalla.
- Compartimentos con pastillas, cada una en el color que dice el panel.
- Horarios correctos en el panel de administración.
- Camino despejado: sin cables sueltos, alfombras gruesas ni desniveles.
- Conexión a internet, si se usará el chat o la voz de ElevenLabs.

7.3 Carga de medicamentos

> **El color en pantalla debe coincidir con el compartimento**
> Cada medicamento se registra con un color (verde, marrón, rojo o morado) y el ESP32 usa ese color para elegir el dispensador. Si la pastilla está en otro compartimento, Pochi entregará un medicamento distinto. Verifique cada color antes de cerrar.

7.4 Apagado

1. Cierre Pochi con el botón ✕ o con `systemctl --user stop pochi`. No lo haga cerca de la hora de un medicamento.
2. Apague la Raspberry desde su menú o con `sudo poweroff`, y espere a que termine.
3. Apague el interruptor de Pochi.

Orden de apagado:

1. *Cierra la aplicación y apaga la batería de los motores (el orden no importa en este paso).*
2. *Apague la batería de los motores (omitir este paso si ya lo hizo en el anterior).*
3. *Apaga la batería de la Raspberry para cortar su energía.*

7.5 Carga de baterías

Use el cargador Steren BR-700 y cargue **una batería a la vez**. Se recomienda cargar todas las noches o cuando no haya entregas próximas.

Cada batería tiene su cargador inteligente. NO cargue las dos baterías al mismo tiempo con el mismo cargador, utilice los dos cargadores para cargar cada batería por separado.

Los cargadores cuentan con ganchos que se deben conectar directamente a las terminales de la batería, rojo para positivo y negro para negativo (rojo con rojo; negro con negro).

Para saber cuanta batería le queda a cada batería en el cargador de cada batería hay una pantalla que al conectar los ganchos en las terminales positivos y negativos se enciende la pantalla e indica cuanta batería queda.

8. Calibración y diagnóstico

8.1 Calibrar la pulsera

Calibre en el lugar real donde se usará Pochi, la primera vez y cada vez que cambie el lugar, la pulsera o su firmware, o si Pochi se detiene lejos, no se detiene o va en la dirección equivocada.

```
systemctl --user stop pochi
cd /home/temario/Documents/interfaz_ia_sistema
source entorno/bin/activate
python3 calibrar_pulsera.py
```

Siga las instrucciones en pantalla. El programa calibra la **distancia** (`RSSI_A_1M_DBM` y `EXPONENTE_TRAYECTO`) y la **dirección** (`OFFSET_DIRECCION_GRADOS`). Al terminar, confirme que los valores quedaron en `robot_movimiento.py` y vuelva a abrir Pochi con `systemctl --user start pochi`.

8.2 Diagnosticar el micrófono

```
python3 probar_microfono.py --todos
```

Muestra el nivel de cada canal, lo que entiende Vosk y si hay bajo voltaje. Causas probables cuando la Raspberry reconoce el micrófono pero no entiende lo que se dice:

- Se empieza a hablar antes de ver "habla ahora" (ya corregido en el programa).
- Está seleccionado otro dispositivo de entrada (`NOMBRE_MICROFONO`).
- El volumen de captura está en 0 o `GANANCIA_MIC` es muy bajo.
- Los canales están invertidos.
- Hay bajo voltaje (revise con `vcgencmd get_throttled`).

8.3 Ver el LiDAR en la laptop

1. Conecte la laptop a la misma red que la Raspberry.
2. En la laptop, ejecute `python3 visor_lidar.py`.
3. En Pochi normal, el mapa aparece solo durante el rastreo. En Pochi LiDAR aparece siempre.
4. La estrella del radar marca la dirección que decidió Pochi según la pulsera.

8.4 Probar los movimientos a mano

Con Pochi cerrado, abra `python3 panel_control.py` para mover brazos, orugas y dispensadores desde la pantalla. También puede usar el Monitor Serie (115200) y escribir los comandos manuales de la sección 5.5.

9. Preparación del show

1. Cargue en el ESP32 el **firmware WiFi** y su página (**Upload** y **Upload Filesystem Image**).
2. Confirme que el nombre y la contraseña de la red en el código son los de la red del show (2.4 GHz).
3. Conecte el teléfono a esa misma red.
4. Encienda a Pochi y lea la IP del ESP32 en el Monitor Serie, o use `http://pochi.local`.
5. En la Raspberry, detenga Pochi normal (`systemctl --user stop pochi`) y abra el ícono "POCHI Show".
6. En el teléfono, abra la IP del ESP32 y pruebe las orugas y los brazos.

7. Haga un ensayo completo: horario de prueba, alerta, dispensación simulada y movimiento desde el teléfono.

> **Seguridad durante el show**
> Las orugas solo se mueven mientras se mantiene presionado el botón en el teléfono, y se detienen solas si se pierde la conexión o si hay un obstáculo a menos de 30 cm. Aun así, mantenga despejado el espacio alrededor de Pochi.

Después del show, vuelva a cargar el **firmware normal** y abra Pochi normal (`systemctl --user start pochi`).

Estado: el firmware WiFi, su página y el modo show se probaron con simuladores y un navegador de prueba. Falta el ensayo completo en el robot real.

Nombre Wifi del show confidencial

10. Solución de problemas

Síntoma	Causa probable	Qué hacer
La Raspberry se reinicia o Pochi se cierra al subir los brazos o al hablar	Bajo voltaje (sección 3.3)	Revise `vcgencmd get_throttled`. Ajuste el convertidor a ~5.1 V, use cables cortos y gruesos o un hub USB con alimentación propia.
Aparece "Low voltage warning" al usar el micrófono	Bajo voltaje	Igual que el caso anterior.
El micrófono no entiende	Dispositivo, volumen, canales o voltaje	Ejecute `probar_microfono.py --todos` (sección 8.2).
Pochi se detiene lejos del paciente o nunca se detiene	Calibración de distancia	Ejecute `calibrar_pulsera.py` en el lugar real.
Pochi va en la dirección equivocada	Calibración de dirección	Ejecute `calibrar_pulsera.py` (ajusta `OFFSET_DIRECCION_GRADOS`).
El punto de la pulsera está gris	La pulsera está apagada, descargada, lejos o con otra MAC	Cárguela y enciéndala, acérquela y revise `TARGET_MAC`.
Pochi no sale a buscar	No detecta la pulsera	Es el comportamiento esperado: muestra la pregunta donde está. Revise la pulsera.
El programa funciona pero no mueve nada	No encontró el ESP32 y entró en simulación	Revise el cable USB y `verificar_puertos.py`; confirme que el ESP32 aparece en `/dev/serial/by-id`.
"No pude darte tu pastilla"	Falla del ESP32 o del dispensador	Revise el compartimento, el servo de ese color y los mensajes `LOG:` en el Monitor Serie o en `journalctl`.
Los brazos suben sin que caiga la pastilla	El infrarrojo está desactivado en el firmware normal	Es el comportamiento actual (suben a los 10 s). Restaurar la lógica del IR cuando se reinstale el sensor.

Síntoma	Causa probable	Qué hacer
Un brazo da un brinco al encender	Reinicio del ESP32 al abrir el puerto	Es normal al arrancar; mantenga las manos alejadas.
El ESP32 no responde a comandos manuales	Hay una dispensación activa	Espere a que termine o envíe `SYS_STOP`.
No se puede abrir el modo show o el LiDAR	Otro modo ya está abierto	Detenga Pochi (`systemctl --user stop pochi`) y cierre los demás programas.
El brillo no cambia	Pantalla que no es DSI o falta la regla udev	Instale la regla udev (sección 6.5).
En el show, las orugas se detienen solas	Seguro de “hombre muerto”, WiFi o freno por obstáculo	Mantenga presionado el botón; revise la red y el espacio libre.
El teléfono no encuentra a Pochi	Otra red o red de 5 GHz	Use la misma red de 2.4 GHz; lea la IP en el Monitor Serie.
“Permission denied” al instalar los íconos	El script no tiene permiso de ejecución	Ejecute `bash modos/instalar_iconos_modos.sh`.
El chat no responde o la voz suena robótica	Sin internet o clave de API inválida	Revise la conexión y las claves. Sin ElevenLabs, Pochi usa espeak.

Si Pochi se traba o deja de funcionar, apague motores, entre al panel de control, haga click en los botones “detener oruga” (el botón rojo de freno de emergencia) y bajar brazos (todo esto sin encender la batería). Revise si hay una desconexión en el cableado interno o algún componente dañado.

Para ponerse en contacto con el soporte técnico (el equipo que creo a Pochi), si es parte del colegio Capouilliez pídale al profesor Alex Godoy el contacto de los creadores.

11. Mantenimiento técnico

- **Pantalla y LiDAR:** pase un paño humedecido con alcohol por la superficie. No aplique el alcohol directamente.
- **Cables y conectores:** revise que estén firmes, sobre todo los de alimentación de la Raspberry y los USB.
- **Baterías:** cárguelas por separado y revise que no estén hinchadas ni con fugas.
- **Respaldo:** guarde una copia de la carpeta `interfaz_ia_sistema` (con `robot.db`) y de los proyectos de firmware antes de cualquier cambio.
- **Actualizaciones:** al recibir una nueva versión del programa, copie solo los archivos que cambiaron y conserve `robot.db`.

Para limpiar los compartimentos y embudo utilice alcohol etílico con un trapito o paño húmedo; humedézcalo abundantemente con el alcohol.

Para limpiar el sensor lidar y pantalla se hace el mismo proceso de limpieza con un trapito y/o paño con el alcohol etílico.

Para limpiar componentes internos y limpiar la placa del circuito esclavo de debe utilizar alcohol isopropílico. NO utiliza agua ni otro tipo de producto de limpieza. Se recomienda que la limpieza de circuitos la haga el soporte técnico o algún experto.

12. Estado del desarrollo (6 de octubre de 2026)

Terminado

- Ciclo de entrega completo, panel de administración, chat con voz, pantalla principal e historial.
- Rastreo guiado por la pulsera, con calibración de distancia y dirección; visor del LiDAR en la laptop.
- Protocolo de energía y brillo; el puerto del ESP32 queda abierto (sin reinicios).
- Firmware: brazos sin brincos y nuevos valores de calibración.
- Firmware WiFi para el show (red compartida, seguridad de orugas) y página del teléfono corregida.
- Modo show, modo LiDAR, prueba de micrófono e íconos de escritorio.

Validado solo en simulación

- Rastreo nuevo (pulsera como guía y vuelta de orientación al llegar) y calibración de dirección.
- Firmware WiFi nuevo y su `index.html`.
- Modo show y modo LiDAR.

Pendiente

1. Correr `calibrar_pulsera.py` (distancia y dirección) en el lugar real.
2. Diagnosticar el micrófono con `probar_microfono.py --todos`.
3. Ensayo completo del show (modo show + teléfono en la misma red).
4. Energía: convertidor a ~5.1 V, cableado corto y grueso o hub USB con alimentación propia.
5. Restaurar la lógica del infrarrojo en el firmware normal cuando se reinstale el sensor.

Anexo A. Referencia rápida de comandos

Raspberry

```
cd /home/temario/Documents/interfaz_ia_sistema
source entorno/bin/activate

python3 main.py                      # Pochi normal (a mano)
python3 modo_show.py                 # Pochi show
python3 modo_lidar.py                # Pochi LiDAR
python3 panel_control.py             # panel manual (Pochi cerrado)
python3 calibrar_pulsera.py          # calibrar la pulsera
python3 probar_microfono.py --todos  # diagnóstico del micrófono

systemctl --user stop pochi          # detener Pochi normal
systemctl --user start pochi         # abrir Pochi normal
journalctl --user -u pochi -f        # mensajes en vivo
vcgencmd get_throttled               # revisar bajo voltaje (0x0 = bien)
bash modos/instalar_iconos_modos.sh  # instalar íconos de escritorio
```

Laptop

```
python3 visor_lidar.py               # radar del LiDAR (UDP 5005)
```

ESP32 (Monitor Serie a 115200)

```
ARM_BOTH_UP        ARM_BOTH_DN        # brazos
MOT_F  MOT_B  MOT_L  MOT_R  MOT_S    # orugas
DISP_TEST_V        DISP_TEST_R        # probar dispensador
SYS_START_V_1                         # ciclo completo: 1 pastilla verde
SYS_RECEIVED                          # confirmar entrega
SYS_STOP                              # detener ciclo
```

Anexo B. Glosario

Término	Significado
BLE	Bluetooth de bajo consumo. La pulsera lo usa para emitir su señal.
RSSI	Intensidad de la señal recibida, en dBm. Mientras más cerca la pulsera, más alta.
LiDAR	Sensor que gira y mide distancias con láser en 360°.
Maestro–esclavo	Un equipo decide (Raspberry) y otro ejecuta (ESP32).
Firmware	Programa que corre dentro del ESP32.
PlatformIO	Extensión de VS Code para compilar y cargar el firmware.
Monitor Serie	Ventana que muestra lo que el ESP32 envía por USB y permite escribirle comandos.
Servo de giro continuo	Servo que gira sin parar mientras recibe la señal; se usa en los dispensadores.
µs (microsegundos)	Unidad del pulso que define la posición de un servo.

Término	Significado
systemd / servicio de usuario	Sistema de Linux que abre Pochi solo al encender.
Entorno virtual	Carpeta `entorno/` con las librerías de Python del programa.
UDP / broadcast	Forma de enviar datos a todos los equipos de la red, sin conexión fija.
WebSocket	Conexión continua entre el teléfono y el ESP32 para mandar órdenes al instante.
Hombre muerto	Seguro que detiene el movimiento si se deja de presionar el botón.
udev	Reglas de Linux que dan permisos a dispositivos; se usa para el brillo.
Vosk	Reconocimiento de voz que funciona sin internet.